Chapter 5

Shared-Memory Programming

Parallel and Distributed Computing

Shared memory makes communication cheap

Threads can read the same array without sending a message.

The same shared address also allows conflicting writes. Correctness now depends on which accesses are ordered.

Today you will reason about shared state

- Classify which objects are shared and which are private.
- Trace a lost update and protect a compound invariant.
- Build a Pthreads example and explain visibility and contention.

Threads share addresses, not execution state



Reachability decides what is shared

Object	Who can reach it?	Question
Global counter	All workers	What protects updates?
Heap array slice	Multiple workers	Are indices disjoint?
Local loop index	Usually one worker	Was its address passed?

One process changes the communication model

MPI ranks own separate address spaces.

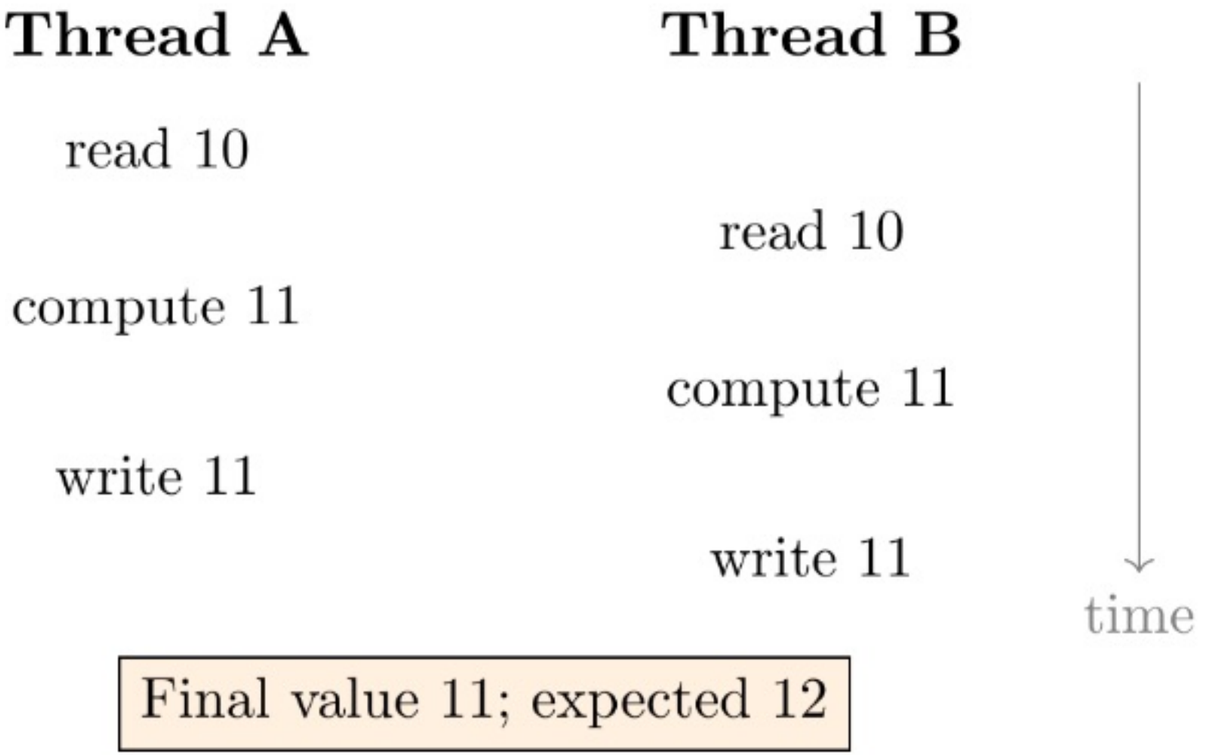
Threads in one process can load and store the same location. This avoids explicit messages but introduces synchronization obligations.

An increment is three steps

```
temporary = counter;
temporary = temporary + 1;
counter = temporary;
```

The source expression counter++ does not make the sequence indivisible.

Two increments can lose one update



One update disappears when both threads store 11.

A data race is undefined in C

- Two threads access one location without required ordering.
- At least one access writes that location.
- A successful test run does not establish correctness.

Atomic access can still violate a rule

Each atomic operation is individually well-defined.

A check followed by a separate update can still let another worker act in between. This is a logical race in the algorithm.

Two sellers can sell one ticket twice

Step	Seller A	Seller B	Tickets
1	Reads 1; allows sale		1
2		Reads 1; allows sale	1
3	Atomic subtract		0
4		Atomic subtract	–1

A critical section protects an invariant

Invariant: balance ≥ 0

The lock must enclose every operation needed to preserve the rule, including the condition check and the balance change.

The bank check and update belong together

```
pthread_mutex_lock(&account_lock);
if (balance >= withdrawal) {
    balance -= withdrawal;
    approved = 1;
}
pthread_mutex_unlock(&account_lock);
```

If balance is 100, two withdrawals of 80 must not both pass.

The lock must cover every related access

- Protect the whole invariant, including relevant reads.
- Keep unrelated computation outside the lock.
- Acquire multiple locks in a consistent order.

Pthreads needs a compiler and one flag

- Include pthread.h; the counter also uses C11 stdatomic.h.
- Compile and link with -pthread.
- Run the executable in the environment that built it.

Linux and macOS setup takes two steps

```
Ubuntu or Debian:
sudo apt update
sudo apt install build-essential

macOS:
xcode-select --install
```

Check the compiler with cc --version after installation.

Windows WSL runs the Linux toolchain

```
Administrator PowerShell:
wsl --install -d Ubuntu

Ubuntu terminal after setup:
sudo apt update
sudo apt install build-essential
```

Build and run the Linux executable inside Ubuntu, not PowerShell.

Windows MSYS2 builds a native executable

```
MSYS2 UCRT64 terminal:
pacman -Syu
pacman -S mingw-w64-ucrt-x86_64-gcc
gcc --version
```

If update closes the shell, reopen UCRT64 and run pacman -Syu again.

Build the first Pthreads example

```
cc -std=c11 -O2 -Wall -Wextra -pthread \
  examples/chapter5/counter_modes.c -o counter_modes
./counter_modes
```

Run from the book project root. MSYS2 uses gcc and counter_modes.exe.

The example has two protected counters

Shared object	Type	Protection
locked_count	ordinary long	count_lock mutex
atomic_count	C11 atomic_long	atomic fetch-add
Expected value	2 × 100,000	200,000

The worker updates both counts safely

```
pthread_mutex_lock(&count_lock);
++locked_count;
pthread_mutex_unlock(&count_lock);
atomic_fetch_add_explicit(
  &atomic_count, 1, memory_order_relaxed);
```

Relaxed ordering is enough for this independent event count.

Main joins before reading results

```
for (int i = 0; i < WORKERS; ++i)
    pthread_create(&threads[i], NULL, worker, NULL);
for (int i = 0; i < WORKERS; ++i)
    pthread_join(threads[i], NULL);
printf(...);
```

After both joins, no worker can change the counts.

The expected count is 200,000

200,000

2 workers × 100,000 iterations

Both counters must equal the independent expected value. Scheduling may vary; the result must not.

Choose synchronization by the invariant

State change	Mechanism	Reason
Independent count	Atomic fetch-add	One object
Check then withdraw	Mutex	One compound operation
Queue entry + length	Mutex	Several fields agree
Immutable lookup	No per-read lock	No concurrent writer

Atomicity does not imply publication

A flag can be atomic while the payload remains unsafe to read.

The consumer needs an ordering relationship with the producer's earlier payload write.

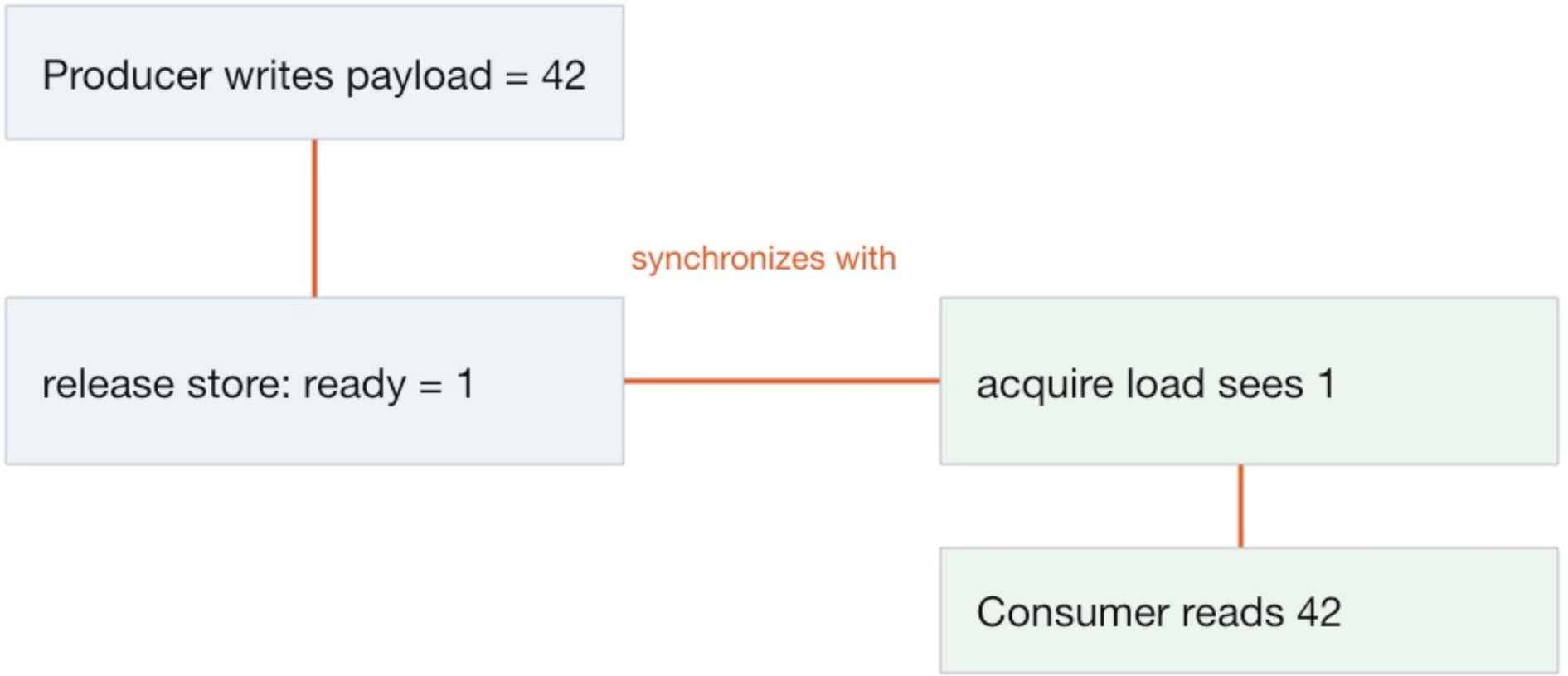
Release and acquire publish the payload

```
payload = 42;
atomic_store_explicit(&ready, 1,
    memory_order_release);

while (!atomic_load_explicit(&ready,
    memory_order_acquire)) { }
answer = payload;
```

The acquire must observe the released value.

Happens-before connects the two threads



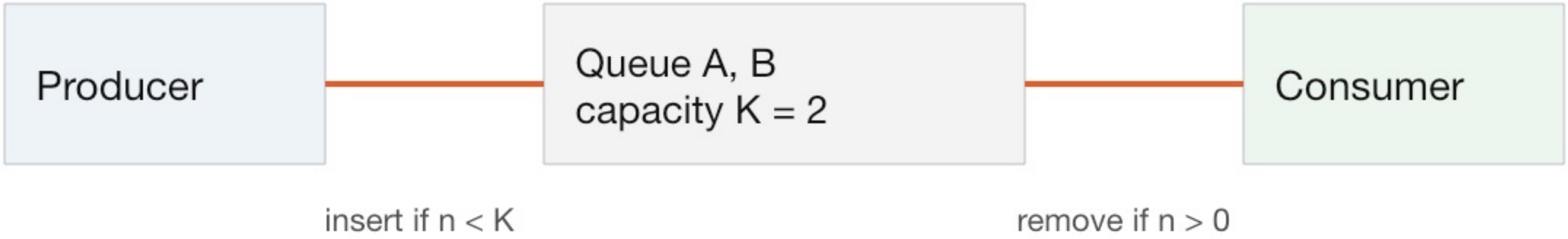
Volatile does not synchronize threads

- volatile does not make counter++ atomic.
- volatile does not publish an ordinary payload.
- Use atomics or a mutex for shared program state.

Each primitive answers a different question

Primitive	Question it answers	Example
Mutex	Who owns this invariant?	Account update
Atomic	Can one value change indivisibly?	Event count
Condition variable	Has a predicate changed?	Queue state
Barrier	Has everyone finished?	Phase boundary

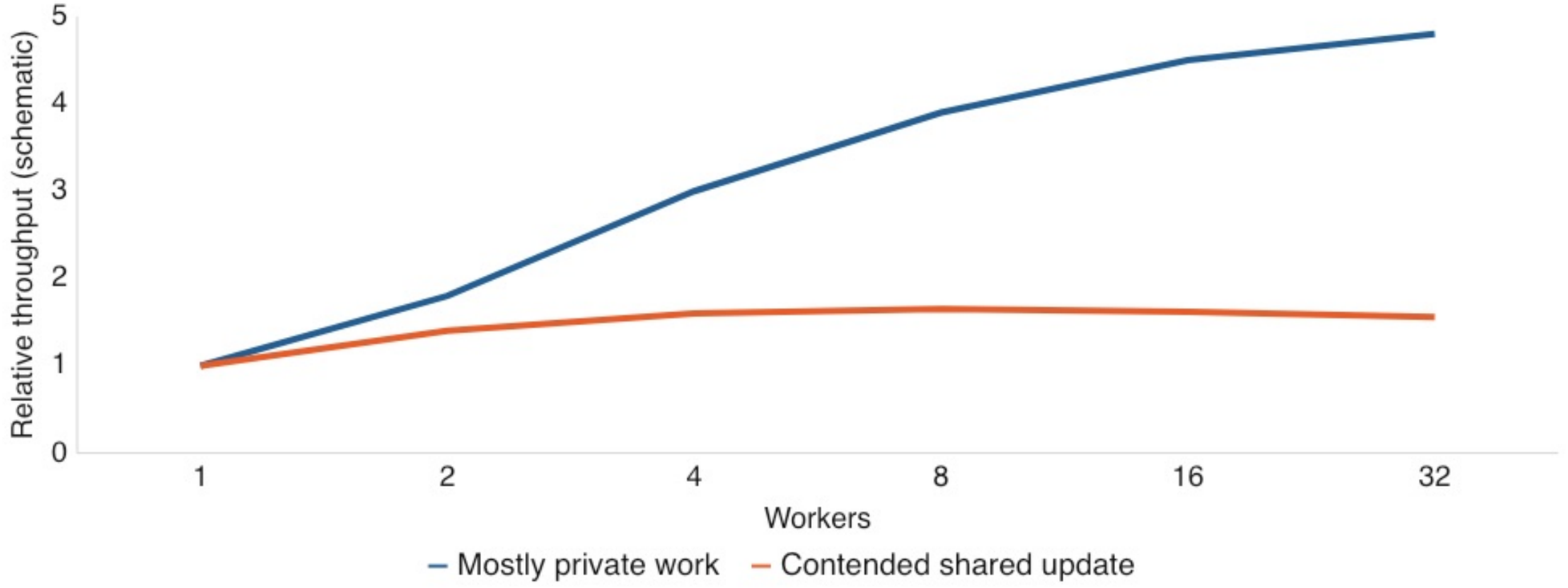
A bounded queue has two predicates



A wakeup does not reserve a queue slot

Step	Queue (K = 2)	Length n	Who may act?
Start	empty	0	producer
Insert A, B	A, B	2	consumer
Remove A	B	1	producer may retry
Insert C	B, C	2	consumer

A shared update can flatten throughput



Schematic trends only — not measured benchmark data

Private subtotals cut shared updates

80,000 → 8

Eight workers × 10,000 records

Per-record shared counting needs 80,000 updates. Private subtotals need eight partial results combined after workers finish.

False sharing differs from a data race

One cache line



Thread 0 writes A

Thread 1 writes B

Different objects; shared cache line

Apply the reasoning to four workers

- Four workers process 250 records each: expected total 1,000.
- Ordinary unsynchronized count: data race.
- Mutex or atomic count: correct; private subtotals: fewer shared updates.

Check your understanding

- Two atomic sellers oversell one ticket. What failed?
- A consumer sees a relaxed ready flag. Is payload published?
- Two counters share a cache line. Is that a data race?

Start with state, rule, then mechanism

- List the exact shared locations and writers.
- State the invariant or publication requirement.
- Choose synchronization, then validate with a small example.